

Lecture 2 - OOP

1) Pointers

NULL pointer - pointing to nowhere.

- 0 or NULL to signal pointers are not currently valid.

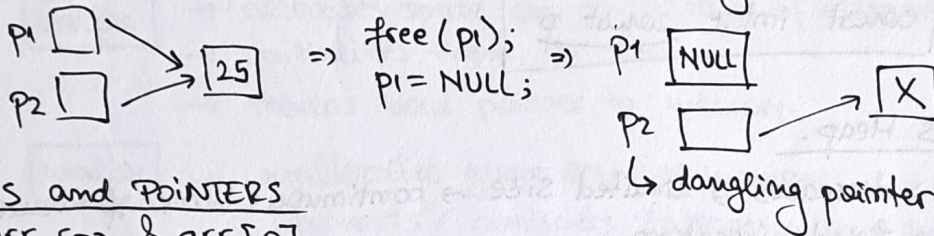
DANGLING pointer - does not point to valid data

- dereference it $\Rightarrow$ undefined behaviour.

- it does point to some memory.

- might print fine because "we are lucky" and $\nexists$ another function to occupy that place in memory.

! Check if a pointer is NULL before dereferencing.



ARRAYS and POINTERS

! arr $\Rightarrow$ &arr[0].

$\hookrightarrow$ arrays can behave as pointers to the first element of the array.
be seen

$\hookrightarrow$ "passed by reference" when passed as function parameters.

FUNCTION POINTER = a pointer which points to an address of a function

- sometimes when we can't pass functions as parameters, we can pass pointers to those functions as parameters.

- functions used as parameters for other functions.

ex: `double (*operation)(double, double);`

`double operationOnVector(vector v, operation  $\theta$ )`

`{ ... }`

ex: • sorting situation (function) as a parameter
• cascading operations

the user can choose the operations
 $\theta = \&\text{sum}$
 $\theta = \&\text{product}$

Ex: For filtering $\rightarrow$ criterion $\rightarrow$ use that criterion defined as a function
pointer $\rightarrow$ pass parameter.

CONST POINTERS

1) changeable pointer to constant data

`const int* p`

$\rightarrow$ the pointed value cannot be changed (*p)
 $\rightarrow$ the pointer can be changed

2) constant pointer to changeable data

`int* const p`

$\rightarrow$ the pointed value can be changed
 $\rightarrow$ the pointer cannot be changed to point to a $\neq$ memory location.

3) constant pointer to constant data

`const int* const p`

Stack vs Heap.

Stack $\rightarrow$ decreases $\rightarrow$ limited size $\rightarrow$ continuous block of memory.
 $\rightarrow$ fixed allocation

Heap $\rightarrow$ increases

$\rightarrow \infty$

$\rightarrow$ de-allocate memory yourself!

$\rightarrow$ heap fragmentation?

$\rightarrow$ dynamic allocation $\rightarrow$ large pool of memory.

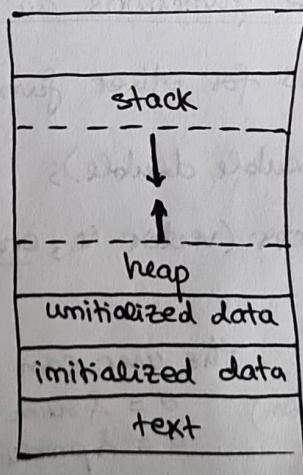
? $\rightarrow$ tries continuous... ! When we need memory that must
"live" longer than a function call.

Ex. of Stack overflow: $\rightarrow$ recursion

$\rightarrow$ `int v[1 000 000]`

$\rightarrow$ can't hold as many integers

high address $\rightarrow$



} command-line arguments
environment variables

} initialized by 0?! (bss)

} read from program file by exec

low address $\rightarrow$

3) Memory allocation

Statically (compile time)

- by declaring variables
- size ^{must} ~~might~~ be known at compile time.
- no control over variable lifetime.

Dynamically (run time)

- on heap
- size does not have to be known ...
- uses pointers
- I control the size and lifetime of variables.

<stdlib.h>

- ↳ **malloc**
 - finds a specified size of free memory
 - returns a void pointer to it
 - uninitialized memory.

- `calloc` → allocates space for an array of elements
- initializes them to zero
- returns void pointer to memory.

realloc → reallocates given area of memory
(expands / contracts / allocates new memory block)

free → releases previously allocated memory.

`int *x = malloc (sizeof (int));` → get a pointer to that memory location if it was successful.

Vector of unknown size:

PAIRS! { `int *x = malloc (sizeof (int) * m);` → allocates as many integers as we've read.
supposing we've read m integers.

free(x); deallocate the memory that we've allocated.

! Otherwise, if we don't use "free" $\Rightarrow$ **memory leaks**. = memory that has been allocated, but not deallocated.

*free(p) $\rightarrow$ freeing only the last integer.

- ↳ free inside loop (for each integer)

delete p1; $\rightarrow$ just one element.

delete [] p3; → an array.

4) Code segment → contains the compiled program

Data segment → stores global + static variables

uninitialised variables → stored in BSS.

Stack → continuous block of memory consisting of stack frames.

→ stores - function parameters

- local variables

- other function-related info.

heap → for dynamically-allocated variables/structures...

• For each function call: - new stack frame is constructed (recursive functions) - pushed onto the stack.

• Function is terminated ⇒ stack frame popped off the stack.

⇒ local var. destroyed

⇒ execution resumed at the return addr.

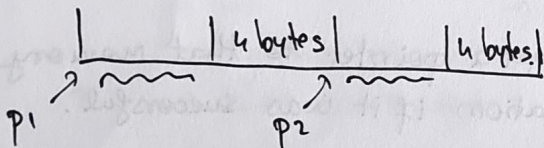
ex: `int a;`

`char x = 'x';`

`int *p = &a;` on stack.

} allocated on the stack and then, they are destroyed.

! Heap fragmentation:



• data of heap → managed by me.

• Size of heap → limited by the size of the virtual memory.

STACK

- fast access
- no need for explicit deallocation
- space managed efficiently
- LIFO; no fragmentation
- local variables only
- stack size → limited
- variables cannot be resized

HEAP

- slow access
- memory managed by programmer.
- efficiency not guaranteed.
- memory may become fragmented.
- vars can be accessed "globally".
- no limit on memory size
- variables can be resized using

realloc()

- C++
- new T

 - memory is allocated to store a value of type T.
 - returns address of a memory location
 - return value has type T^* .

delete T

 - deallocates the memory allocated using new.
 - precondition: p is of type T^* .
 - memory space allocated to the variable p is free.

`<crtdbg.h>` → `_CrtDumpMemoryLeaks()`

6) Memory errors:

- Invalid memory access (unallocated / deallocated memory accessed)
- Mismatched Allocation / Deallocation
- Freeing memory that was never allocated.
- Repeated frees (freeing sth that was already freed)

7) When to use? what??

- 1) allocate data on heap.
- 2) lifetime of variable > that function
- 3) avoid copying data.

```
{
    int vector[10000];
    int a = 10;
}
```

→ $f(\text{Vector } *v)$ more space-efficient than doing $f(\text{Vector } v)$. **!!**

8) Modules. = collection of functions and variables that implements a well-defined functionality.

→ to separate interface from implementation.

9) Header files. → function declarations are grouped into a separate file called header file.

10) Library = set of functions, exposed for use by other programs.

└→ .h → function prototypes

└→ .dll, .lib → compiled implementation.

1) Source code (.c / .cpp) does not need to be shared.

! Declaration in header, implementation in .c / .cpp

2) math.h → header file.

#include "math.h" → preprocessor directive.

↳ "open that file and insert its contents".

#include <header_file>

↳ searched in the system directories.

#include "header_file"

↳ searched in the current directory.

↳ then in the system directory.

#define identifier replacement

↳ "any occurrence of identifier in the code is replaced by replacement".

#ifdef macro, ..., #endif ...

#ifndef macro, ..., #endif ...

#pragma once

↳ preprocessor directive

what file?

↳ makes sure a specific file will be contained only once in the translation unit

? "single compilation

13)

→ interface

.h → function declarations. → client → define the "WHAT".

.c / .cpp → function implementations. → hidden
→ define the "HOW".

→ ADT implementation in C/C++.